Huffman Compression and Decompression Assignment Report

1. Introduction

Data compression is an essential technique used to reduce the size of data for efficient storage and faster transmission. Compression algorithms are generally categorized into two main types:

- **Lossless Compression** – No information is lost during compression and the original data can be perfectly reconstructed.
- **Lossy Compression** – Some information is permanently removed to achieve higher compression ratios.

This project implements the **Huffman Compression Algorithm**, a well-known **lossless compression technique** introduced by David A. Huffman in 1952.

Huffman coding is widely used in many compression systems such as **ZIP files, JPEG images, and multimedia formats**.

The main objectives of this project are:

- Implement **Huffman encoding (compression)**
- Implement **Huffman decoding (decompression)**
- Generate Huffman codes based on character frequency
- Verify correctness by ensuring the decoded output matches the original input
- Evaluate compression efficiency using different test cases

---

2. Overview of Huffman Algorithm

Huffman coding is a **frequency-based compression algorithm** that assigns **shorter binary codes to frequently occurring characters** and **longer codes to less frequent characters**.

The algorithm builds a **binary tree called the Huffman Tree**, where:

- Each **leaf node** represents a character
- Each **internal node** represents the combined frequency of its children

Characters with **higher frequency** are placed closer to the root, resulting in **shorter codes**.

Each character is encoded as a binary sequence generated by traversing the Huffman tree:

- **Left branch → 0**
- **Right branch → 1**

This method ensures that the final encoding is **prefix-free**, meaning no code is a prefix of another.

---

3. How the Algorithm Works

3.1 Compression Process

The compression process follows these steps:

1. Read the **input text file**.
2. Calculate the **frequency of each character**.
3. Create a **priority queue** containing nodes for each character.

4. Build the **Huffman Tree** by repeatedly:
    - Extracting two nodes with the smallest frequency
    - Creating a new parent node with their combined frequency
5. Continue until a **single root node** remains.
6. Traverse the tree to generate **binary Huffman codes** for each character.
7. Encode the input text using these codes.
8. Save:
    - The compressed binary data in **encoded.bin**
    - The Huffman codes in **Huffman_Codes.txt**

3.2 Decompression Process

The decompression process reconstructs the original text using the Huffman codes.

Steps:

1. Read the **encoded binary file**.
2. Load the **Huffman codes dictionary**.
3. Rebuild the **Huffman Tree** using the stored codes.
4. Traverse the tree according to the binary sequence:
    - `0 → left`
    - `1 → right`
5. When a leaf node is reached, output the corresponding character.
6. Continue until the entire encoded sequence is processed.
7. Write the result into **decoded.txt**.

The decompressed text should **exactly match the original input text**.

---

4. Implementation

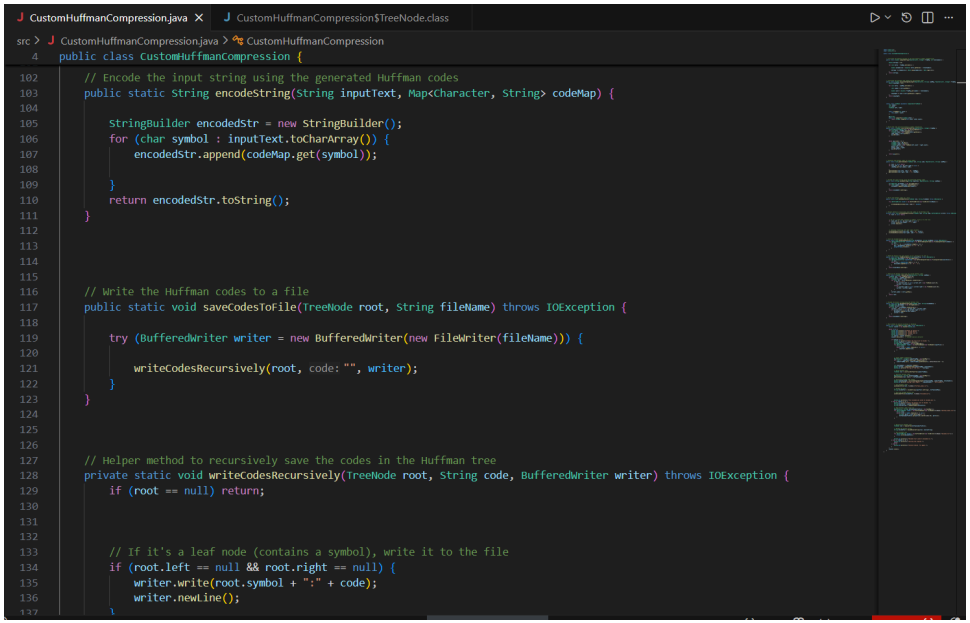
4.1 Compression Code

Figure 1: Huffman Compression Implementation

Implementation details:

- Programming Language: **Java**
- Tree Structure: **Custom TreeNode class**
- Priority Queue used for building Huffman Tree
- Character frequencies calculated from **input.txt**
- Huffman codes stored in **Huffman_Codes.txt**
- Encoded binary output saved in **encoded.bin**

The encoder reads the input file, calculates character frequencies, builds the Huffman tree, generates binary codes, and compresses the text.



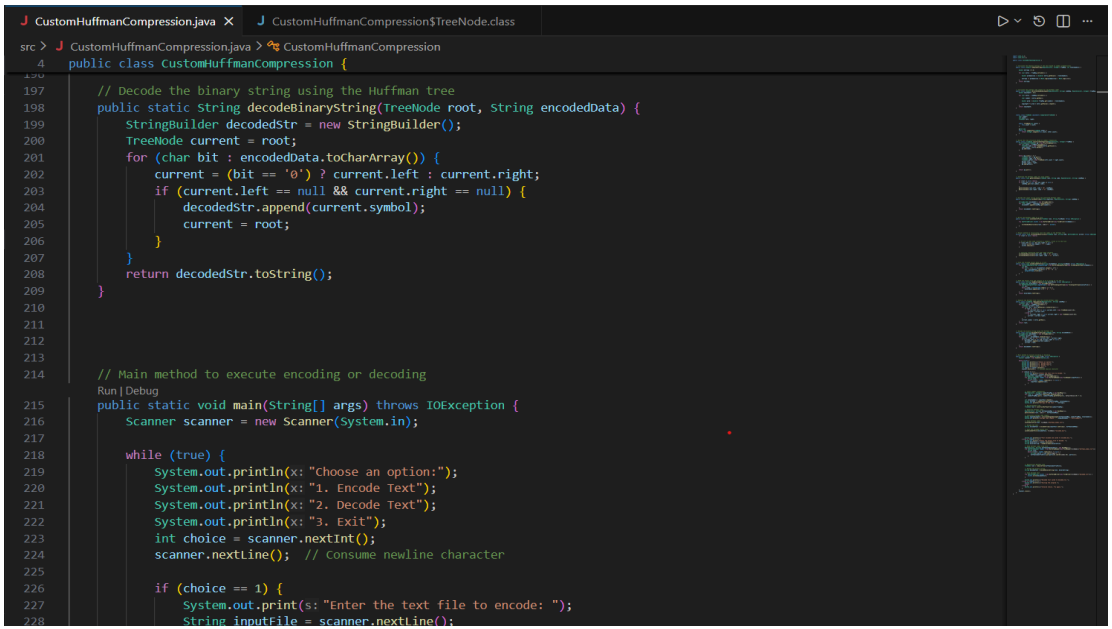
4.2 Decompression Code

Figure 2: Huffman Decompression Implementation

Implementation details:

- Reads compressed data from **encoded.bin**
- Uses **Huffman_Codes.txt** to rebuild the tree
- Decodes binary sequence into characters
- Writes reconstructed text into **decoded.txt**

The decoder ensures that the decompressed text **matches the original input exactly**.



---

5. Results and Case Studies

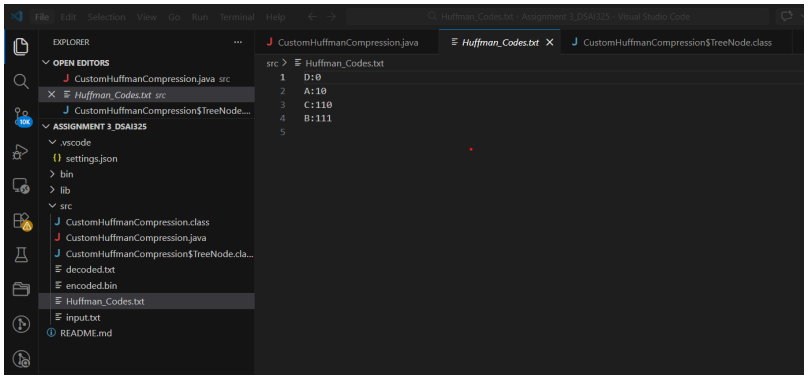
Case Study 1

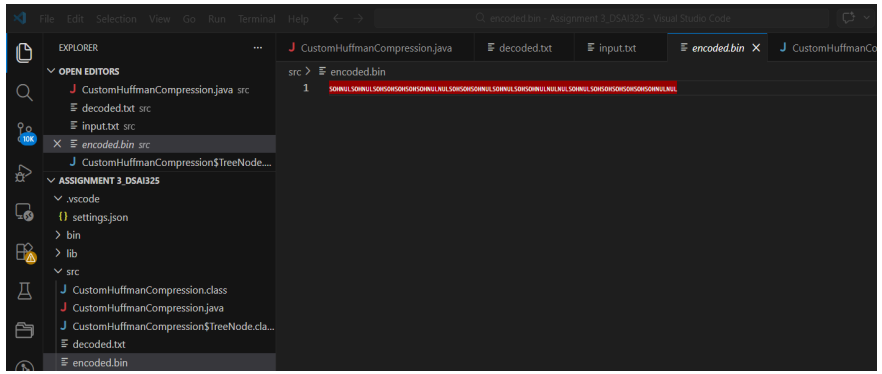
Input: AABCDBDACDDABBDD

Original Length: 14 characters

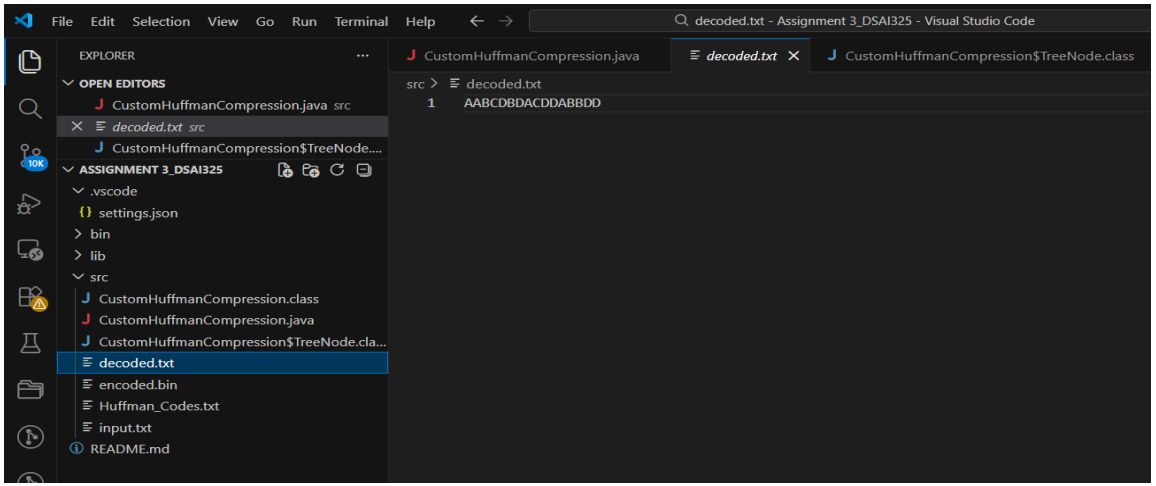
Expected Behavior: Since characters **a** and **b** appear frequently, the algorithm should assign **shorter binary codes** to them.

Compressed Output: Binary sequence stored in **encoded.bin**





Decompressed Output:



Correctness : Decompressed == Original → **Yes**

Observation : Compression works well because repeated characters receive shorter binary codes.

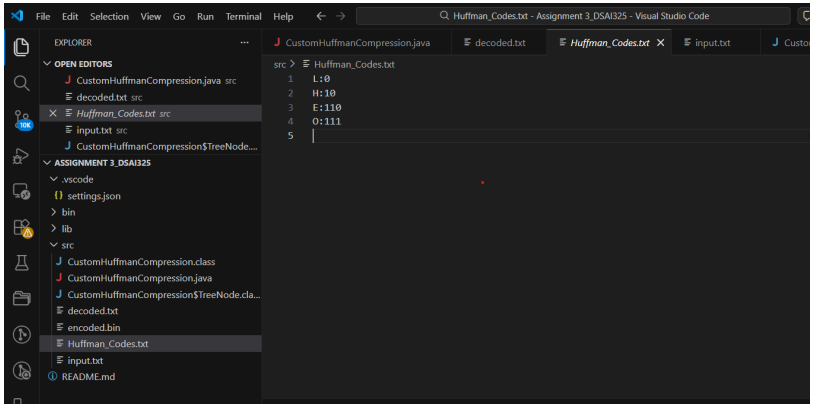
Case Study 2

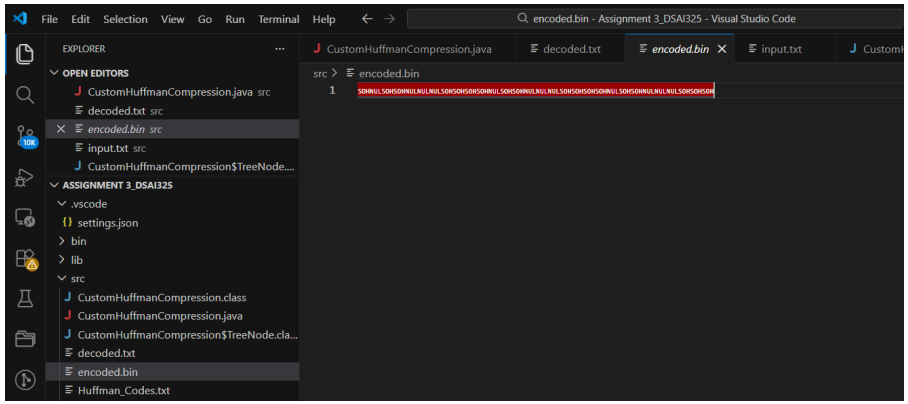
Input: HELLOHELLOHELLO

Original Length: 15 characters

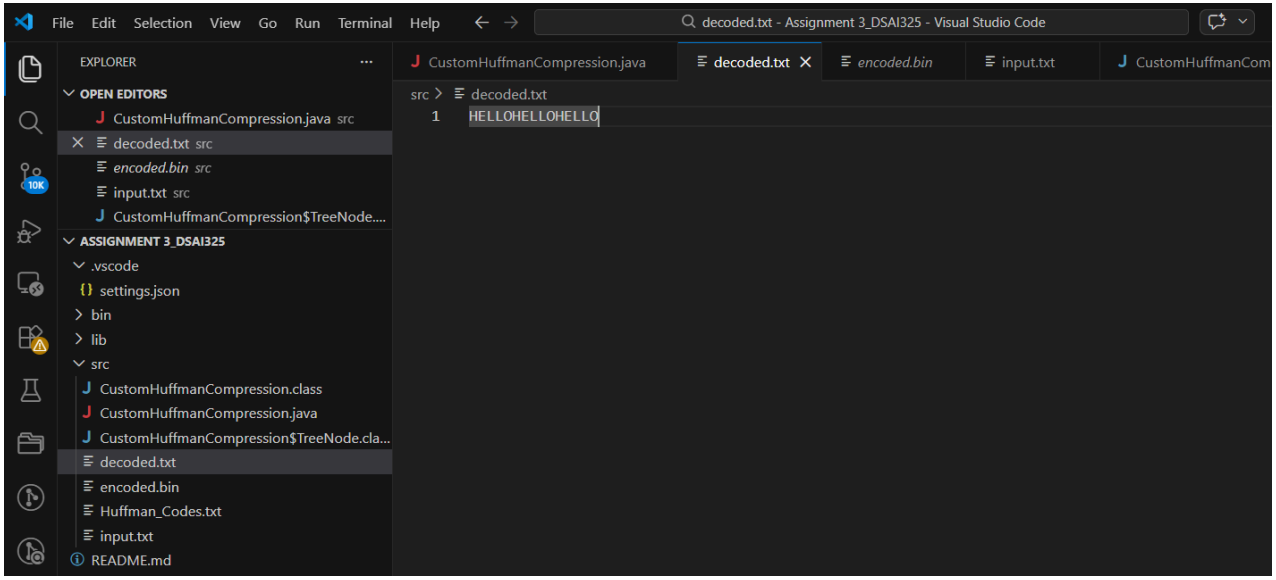
Expected Behavior: Characters such as **h, e, l, o** appear multiple times. Huffman coding should assign shorter codes to the most frequent characters.

Compressed Output: Stored in **encoded.bin**





Decompressed Output:



Correctness: Decompressed == Original → **Yes**

Observation: Compression improves when characters repeat frequently because high-frequency characters receive shorter binary codes.

---

6. Advantages and Limitations

Advantages

- **Lossless compression** (no information loss)
- Efficient for text with **repeated characters**
- Produces **optimal prefix codes**
- Widely used in real-world compression systems
- Relatively simple to implement

Limitations

- Requires storing the **Huffman tree or codes** for decoding
- Less efficient when character frequencies are similar
- Tree construction adds **computational overhead**
- Not ideal for very small datasets

---

7. Conclusion

This project successfully implemented the **Huffman Compression and Decompression algorithm** in Java.

Key findings:

- Huffman coding effectively reduces data size by assigning shorter codes to frequently occurring characters.
- The algorithm dynamically builds a tree based on character frequencies.
- The decompression process perfectly reconstructs the original text.
- The method is completely **lossless**, meaning no information is lost.